

DOCUMENTACION TECNICA

Libreria de Poses MuJoCo para Unitree R1

Paquete tecnico MUJOCO R1 para simulacion, captura y ejecucion de rutinas

Campo	Detalle
Paquete / Script	MUJOCO R1; script principal r1_pose_player_mujoco.py
Version	1.0
Plataforma	Ubuntu 20.04 recomendado; Python 3; Unitree MuJoCo
Robot / Hardware	Unitree R1 en simulacion MuJoCo
SDK / Framework	Unitree SDK2 Python; DDS; LowCmd / Low- State
Script base	unitree_mujoco con modelo R1 corregido
Autor	Robotics 4.0
Fecha	Julio de 2026

0. Contenido

1	Descripcion General	3
1.1	Componentes desarrollados	4
1.2	Usuarios previstos	4
2	Arquitectura del Sistema	4
2.1	Capas funcionales	5
2.2	Comunicacion	5
2.3	Diagrama textual de flujo	5
3	Requisitos y Dependencias	6
3.1	Requisitos principales	6
3.2	Verificacion de unitree_sdk2py	6
3.3	Variable LD_LIBRARY_PATH	7
4	Organizacion de Archivos	7
4.1	Estructura esperada	7
4.2	Archivos obligatorios	8
4.3	Rutas recomendadas	8
5	Formato de Entrada	8
5.1	Campos de una rutina	8
5.2	Indices validos	9
5.3	Pose segura obligatoria	10
5.4	Ejemplo minimo funcional	10
6	Codigo Fuente y Funcionamiento Interno	11
6.1	Script principal: r1_pose_player_mujoco.py	11
6.1.1	Responsabilidades principales	11
6.1.2	Grupos articulares usados	11
6.1.3	Ganancias finales	12
6.1.4	Rampa de activacion	12
6.2	Script de captura: r1_capture_pose.py	12
6.3	Modelo corregido: r1.xml	13
7	Flujo de Ejecucion	13
7.1	Paso 1: ubicar la carpeta descargada	13
7.2	Paso 2: verificar Unitree MuJoCo	13
7.3	Paso 3: instalar archivos R1 finales dentro de Unitree MuJoCo	14
7.4	Paso 4: verificar unitree_sdk2py	14
7.5	Paso 5: abrir el simulador R1	14
7.6	Paso 6: ejecutar una rutina	14
7.7	Paso 7: validar sintaxis de una rutina	15
7.8	Paso 8: capturar una nueva pose	15

8 Seguridad Operacional	15
8.1 Condiciones antes de ejecutar	15
8.2 Que no debe modificarse sin precaucion	16
8.3 Detencion en simulacion	16
8.4 Consideraciones para robot fisico	16
9 Guia de Uso Rapido	16
9.1 Ejecucion rapida de siuu	16
9.2 Ejecucion rapida de dab	17
10 Problemas Conocidos y Soluciones	17
11 Extension y Mantenimiento	18
11.1 Agregar una nueva rutina	18
11.2 Recomendaciones para rutinas nuevas	18
11.3 Validaciones despues de modificar	18
12 Correcciones Tecnicas Realizadas	18
13 Resumen de Actividades	20



1. Descripcion General

El paquete MUJOCO R1 es una entrega tecnica organizada para ejecutar, capturar y extender rutinas de movimiento de cintura, brazos y cabeza del robot Unitree R1 dentro del entorno Unitree MuJoCo. Esta documentacion parte de un escenario concreto: el usuario recibe una carpeta llamada MUJOCO R1 y ya cuenta, o debe instalar previamente, una copia funcional del repositorio `unitree_mujoco` y del paquete `unitree_sdk2py`.

La herramienta permite ejecutar rutinas de ejemplo como `dab.txt` y `siuu.txt`, capturar nuevas poses desde el simulador y crear nuevas rutinas en archivos `.txt` con formato JSON. El flujo esta pensado para equipos tecnicos que necesitan replicar el entorno R1 sin revisar todo el repositorio de Unitree, que contiene tambien otros robots y recursos no necesarios para esta entrega.

Durante el desarrollo se corrigieron tres fallas criticas:

1. El modelo R1 no tenia sensores articulares suficientes para que `LowState` entregara posiciones, velocidades y torques estimados en el orden esperado por el bridge.
2. Los actuadores heredaban inicialmente un `ctrlrange` demasiado bajo, por lo que los brazos no alcanzaban poses amplias.
3. Al ampliar la autoridad de control sin amortiguacion, el robot presentaba oscilaciones rapidas. Esto se estabilizo con limites por grupo y rampa progresiva de ganancias.



1.1 Componentes desarrollados

Componente	Tipo	Descripcion	Estado
r1_pose_player_mujoco.py	Script Python	Ejecuta rutinas TXT/JSON sobre el R1 mediante LowCmd. Incluye mapeo por Actuator_index, interpolacion y rampa de ganancias.	Final
r1_capture_pose.py	Script Python	Captura la pose actual desde LowState y genera un bloque JSON listo para pegar en una rutina.	Final
r1.xml	Modelo MuJoCo	Modelo R1 corregido con sensores articulares y limites de control ajustados.	Final
config.yaml	Configuracion	Selecciona robot R1, interfaz io, dominio 0 y elastic_band.	Final
dab.txt	Rutina TXT	Rutina de ejemplo tipo dab. Inicia y termina en pose segura.	Validada
siuu.txt	Rutina TXT	Rutina de ejemplo con dos movimientos y sostenimiento de pose.	Validada
r1_safe_pose_final.json	Configuracion	Define la pose segura estandar para iniciar y finalizar rutinas.	Final
r1_joint_map_final.json	Referencia	Documenta indices validos para cintura, brazos y cabeza.	Final

1.2 Usuarios previstos

- Ingenieros o tecnicos que deban entregar un entorno R1 reproducible.
- Usuarios que ya tengan instalado unitree_mujoco y quieran ejecutar poses R1.
- Desarrolladores que necesiten crear nuevas rutinas de brazos, cabeza y cintura.
- Equipos que quieran usar dab.txt y siuu.txt como ejemplos base.

2. Arquitectura del Sistema

La arquitectura se basa en un computador local ejecutando Unitree MuJoCo y scripts Python que publican comandos de bajo nivel mediante DDS. El simulador representa el robot R1 y expone topicos compatibles con el flujo LowCmd/LowState. El script principal lee rutinas desde archivos .txt, interpola posiciones objetivo y envia comandos articulares al simulador.

2.1 Capas funcionales

Capa / Modulo	Funcion	Entrada	Salida
Modelo R1 MuJoCo	Representa cinematica, dinamica, actuadores y sensores del R1.	r1.xml, scene.xml, meshes	Estado fisico simulado
Simulador Unitree MuJoCo	Carga el robot, ejecuta la simulacion y conecta DDS.	config.yaml	Topicos LowState/LowCmd
Script de ejecucion	Lee rutinas, interpola pasos y publica comandos.	dab.txt, siuu.txt u otra rutina	Movimiento del R1 en MuJoCo
Script de captura	Lee la pose actual desde LowState.	Pose actual del robot en simulador	Bloque JSON de posiciones
Rutinas TXT	Definen secuencias temporales de posiciones articulares.	JSON en archivo .txt	Objetivos articulares en radianes

2.2 Comunicacion

Canal / Interfaz	Uso	Entorno	Observacion
DDS sobre lo	Comunicacion local entre Python y MuJoCo.	Simulacion local	Puede mostrar advertencia de multicast en lo; no necesariamente bloquea el flujo local.
rt/lowcmd	Envio de comandos articulares al simulador.	Unitree SDK2 / MuJoCo	El script publica posiciones, ganancias y amortiguamiento.
rt/lowstate	Lectura de estados articulares.	Unitree SDK2 / MuJoCo	Requiere sensores jointpos, jointvel y actuatorfrq.
config.yaml	Define robot, escena, dominio e interfaz.	Simulador	Debe apuntar a robot: "r1" e interface: "lo".

2.3 Diagrama textual de flujo

```

MUJOCO R1/
|
|-- 04_rutinas_txt/dab.txt
|-- 04_rutinas_txt/siuu.txt
|
v

```

```
03_scripts_movimiento/r1_pose_player_mujoco.py
|
|-- lee rutina
|-- valida indices 12 a 25
|-- interpola posiciones
|-- aplica rampa de ganancias
v
Unitree SDK2 Python / DDS
|
|-- publica: rt/lowcmd
|-- recibe: rt/lowstate
v
Unitree MuJoCo
|
v
Modelo R1 corregido: r1.xml
```

3. Requisitos y Dependencias

El paquete presupone que el computador ya cuenta con una instalacion funcional de Unitree MuJoCo. La carpeta MUJOCO R1 no reemplaza todo el repositorio original de Unitree; entrega los archivos R1 finales y los scripts de poses.

3.1 Requisitos principales

Requisito	Version / Valor	Verificacion
Sistema operativo	Ubuntu 20.04 recomendado	lsb_release -a
Python	Python 3	python3 --version
Unitree MuJoCo	Repositorio funcional en \$HOME/unitree_mujoco	ls \$HOME/unitree_mujoco
Unitree SDK2 Python	Paquete unitree_sdk2py importable	python3 -c "import unitree_sdk2py; print('OK')"
MuJoCo library path	Ruta a libmujoco.so.3.2.7	find \$HOME/unitree_mujoco -name "libmujoco*"
Interfaz local	lo para simulacion local	ip link show lo
Modelo R1 corregido	r1.xml final del paquete	grep -n "jointpos" r1.xml
Config simulador	robot: "r1" y enable_elastic_band: 1	cat simulate/config.yaml

3.2 Verificacion de unitree_sdk2py

Ejecutar:

```
python3 --version
python3 -c "import unitree_sdk2py; print('unitree_sdk2py OK')"
```

Si aparece:

```
ModuleNotFoundError: No module named 'unitree_sdk2py'
```

instalar o exponer correctamente el paquete de Unitree SDK2 Python. Una instalacion tipica desde el repositorio local se realiza asi:

```
cd ~/unitree_sdk2_python
pip3 install -e .
```

Si el repositorio esta en otra ruta, cambiar ~/unitree_sdk2_python por la ruta real.

3.3 Variable LD_LIBRARY_PATH

Para evitar errores de libreria MuJoCo, ejecutar:

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HOME/unitree_mujoco/simulate/mujoco/build/lib:/opt/
unitree_robotics/lib
```

Para dejarlo permanente:

```
echo 'export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HOME/unitree_mujoco/simulate/mujoco/build/lib:/opt/
unitree_robotics/lib' >> ~/.bashrc
source ~/.bashrc
```

4. Organizacion de Archivos

La entrega debe iniciar desde una carpeta llamada MUJOCO R1. Esta carpeta contiene los archivos especificos y finales del flujo R1.

4.1 Estructura esperada

```
MUJOCO R1/
|-- 01_modelo_r1_final/
|   '-- unitree_robots/
|       '-- r1/
|           |-- r1.xml
|           |-- scene.xml
|           '-- meshes y assets del modelo R1
|
|-- 02_config_simulador/
|   '-- config.yaml
|
|-- 03_scripts_movimiento/
|   |-- r1_pose_player_mujoco.py
|   '-- r1_capture_pose.py
|
|-- 04_rutinas_txt/
|   |-- test_hold_actual.txt
|   |-- dab.txt
|   '-- siuu.txt
|
|-- 05_config_r1/
|   |-- r1_joint_map_final.json
|   '-- r1_safe_pose_final.json
|
```



```
|-- 06_herramientas_diagnostico/
|   '-- scripts auxiliares de prueba
|
|-- 07_instalacion/
|   '-- instalar_archivos_r1_en_unitree_mujoCo.sh
|
|-- run_simulador_r1.sh
|-- run_dab.sh
|-- run_siuu.sh
|-- capturar_pose_r1.sh
'-- README.md
```

4.2 Archivos obligatorios

Archivo	Uso	Obligatorio
01_modelo_r1_final/unitree_robot	Modelo R1 corregido.	Si
01_modelo_r1_final/unitree_robot	Escena principal del R1.	Si
02_config_simulador/config.yar	Configuracion final para abrir R1 en MuJoCo.	Si
03_scripts_movimiento/r1_pose_r1.py	Diagrama de rutinas.	Si
03_scripts_movimiento/r1_captu	Captura nuevas poses.	Recomendado
04_rutinas_txt/*.txt	Rutinas de movimiento.	Si
07_instalacion/instalar_archiv	Copia modelo y configuracion al repositorio Unitree MuJoCo.	Si

4.3 Rutas recomendadas

Se recomienda ubicar la carpeta en:

```
$HOME/Projects/unitree_lab/MUJOCO R1
```

El nombre contiene un espacio. Los scripts incluidos usan comillas internamente para soportarlo. Si se escriben comandos manuales, usar comillas:

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"
```

5. Formato de Entrada

Las rutinas estan escritas como JSON valido dentro de archivos .txt. Cada rutina contiene metadatos generales y una lista ordenada de pasos. Cada paso define un nombre, un diccionario de posiciones articulares y una duracion en segundos.

5.1 Campos de una rutina

Campo	Tipo	Obligatorio	Descripcion
nombre_rutina	string	Si	Nombre logico de la rutina.
fecha_creacion	string	Si	Fecha de creacion o actualizacion.
descripcion	string	Si	Explicacion breve del movimiento.
numero_pasos	entero	Si	Numero total de pasos en la lista pasos.
pasos	lista	Si	Secuencia ordenada de posiciones.
nombre	string	Si	Nombre descriptivo de cada paso.
posiciones	objeto	Si	Pares "indice": valor en radianes.
duracion	numero	Si	Tiempo de transicion o sostenimiento en segundos.

5.2 Indices validos

Las posiciones usan `Actuator_index`, no `Joint_index`. Para rutinas de pose se deben usar los indices 12 a 25.

Indice	Articulacion
12	<code>waist_roll_joint</code>
13	<code>waist_yaw_joint</code>
14	<code>left_shoulder_pitch_joint</code>
15	<code>left_shoulder_roll_joint</code>
16	<code>left_shoulder_yaw_joint</code>
17	<code>left_elbow_joint</code>
18	<code>left_wrist_roll_joint</code>
19	<code>right_shoulder_pitch_joint</code>
20	<code>right_shoulder_roll_joint</code>
21	<code>right_shoulder_yaw_joint</code>
22	<code>right_elbow_joint</code>
23	<code>right_wrist_roll_joint</code>
24	<code>head_pitch_joint</code>
25	<code>head_yaw_joint</code>

Advertencia critica. No usar indices 26 o superiores en rutinas R1. El mensaje `LowState` puede tener mas espacios internos por estructura IDL, pero el R1 activo en esta simulacion usa 26 actuadores reales, indexados de 0 a 25.

5.3 Pose segura obligatoria

Todas las rutinas deben iniciar y terminar en la siguiente pose segura:

```
{  
  "12": 0.0,  
  "13": 0.0,  
  "14": 0.0,  
  "15": 0.151786,  
  "16": 0.0,  
  "17": 1.062987,  
  "18": 0.0,  
  "19": 0.0,  
  "20": -0.151872,  
  "21": 0.0,  
  "22": 1.062987,  
  "23": 0.0,  
  "24": 0.0,  
  "25": 0.0  
}
```

5.4 Ejemplo minimo funcional

```
{  
  "nombre_rutina": "ejemplo_minimo",  
  "fecha_creacion": "2026-07-03",  
  "descripcion": "Rutina minima R1 con pose segura.",  
  "numero_pasos": 2,  
  "pasos": [  
    {  
      "nombre": "Pose segura inicial",  
      "posiciones": {  
        "12": 0.0,  
        "13": 0.0,  
        "14": 0.0,  
        "15": 0.151786,  
        "16": 0.0,  
        "17": 1.062987,  
        "18": 0.0,  
        "19": 0.0,  
        "20": -0.151872,  
        "21": 0.0,  
        "22": 1.062987,  
        "23": 0.0,  
        "24": 0.0,  
        "25": 0.0  
      },  
      "duracion": 1.0  
    },  
    {  
      "nombre": "Pose segura final",  
      "posiciones": {  
        "12": 0.0,  
        "13": 0.0,  
        "14": 0.0,  
        "15": 0.151786,  
        "16": 0.0,  
        "17": 1.062987,  
        "18": 0.0,  
        "19": 0.0,  
        "20": -0.151872,  
        "21": 0.0,  
        "22": 1.062987,  
        "23": 0.0,  
        "24": 0.0,  
        "25": 0.0  
      },  
      "duracion": 1.0  
    }  
  ]  
}
```

```

    "19": 0.0,
    "20": -0.151872,
    "21": 0.0,
    "22": 1.062987,
    "23": 0.0,
    "24": 0.0,
    "25": 0.0
  },
  "duracion": 1.0
}
]
}

```

6. Codigo Fuente y Funcionamiento Interno

6.1 Script principal: `r1_pose_player_mujoco.py`

Este script es el ejecutor de rutinas. Recibe como argumento la ruta de una rutina .txt, carga el JSON, valida los pasos, mantiene las piernas en una postura estable inicial y controla cintura, brazos y cabeza segun los objetivos de la rutina.

6.1.1 Responsabilidades principales

Elemento	Responsabilidad	Entradas	Salidas / Efecto
Constantes R1	Definen cantidad de motores y grupos articulares.	Codigo fuente	Mapeo estable de articulaciones
<code>gains_for_joint(i)</code>	Retorna K_p y K_d por articulacion.	Indice de actuador	Ganancias de control
Carga de rutina	Lee y valida archivo JSON/TXT.	Ruta del archivo	Lista de pasos
Interpolacion	Calcula transicion gradual entre poses.	Pose inicial, pose objetivo, tiempo	Comando articular intermedio
Rampa de ganancias	Evita activar K_p/K_d de golpe.	Tiempo desde inicio	Escala progresiva de control
Publicacion DDS	Envia LowCmd.	Posiciones y ganancias	Movimiento en MuJoCo
Cierre seguro	Detiene hilo recurrente y termina proceso.	Final de rutina o interrupcion	Control detenido

6.1.2 Grupos articulares usados

```

ACTIVE_MOTOR_COUNT = 26

LEG_JOINTS = list(range(0, 12))

```

```
WAIST_JOINTS = [12, 13]
LEFT_ARM_JOINTS = [14, 15, 16, 17, 18]
RIGHT_ARM_JOINTS = [19, 20, 21, 22, 23]
HEAD_JOINTS = [24, 25]

POSE_JOINTS = WAIST_JOINTS + LEFT_ARM_JOINTS + RIGHT_ARM_JOINTS + HEAD_JOINTS
```

6.1.3 Ganancias finales

Las ganancias finales se ajustaron para eliminar twitching rapido y conservar suficiente autoridad de movimiento en brazos.

```
def gains_for_joint(i):
    """
    Ganancias R1 finales.
    Objetivo: estabilidad en hold y movimientos suaves en rutinas.
    """

    if i in LEG_JOINTS:
        return 12.0, 0.8

    if i in WAIST_JOINTS:
        return 15.0, 1.0

    if i in [14, 15, 16, 17, 19, 20, 21, 22]:
        return 14.0, 1.2

    if i in [18, 23]:
        return 8.0, 0.8

    if i == 24:
        return 15.0, 0.8

    if i == 25:
        return 6.0, 0.4

    return 0.0, 0.0
```

6.1.4 Rampa de activacion

La rampa de activacion fue necesaria porque el control entraba de forma brusca y generaba oscilaciones. El factor de escala inicia en 0 y llega a 1 en aproximadamente 2.5 segundos.

```
def gain_scale(self):
    if self.writer_start_time is None:
        return 0.0

    elapsed = time.time() - self.writer_start_time
    return max(0.0, min(elapsed / self.gain_ramp_duration, 1.0))
```

6.2 Script de captura: r1_capture_pose.py

El script de captura lee LowState, imprime las posiciones actuales de los indices 12 a 25 y genera un bloque JSON listo para pegar en una rutina.

Flujo de captura:

1. Abrir MuJoCo con el R1.
2. Pausar la simulacion si se desea mover sliders manualmente.
3. Posicionar cintura, brazos y cabeza.
4. Ejecutar `r1_capture_pose.py`.
5. Copiar el bloque JSON generado.

6.3 Modelo corregido: `r1.xml`

El modelo final contiene tres grupos de sensores en orden compatible con el bridge de Unitree MuJoCo:

1. `jointpos` para los 26 actuadores.
2. `jointvel` para los 26 actuadores.
3. `actuatorfrc` para los 26 actuadores.

Nota tecnica. El bridge usa `sensordata[i]` como posicion, `sensordata[i + num_motor]` como velocidad y `sensordata[i + 2*num_motor]` como torque estimado. Si el XML no tiene los sensores en ese orden, el robot recibe estados incorrectos y se contorsiona.

7. Flujo de Ejecucion

Esta seccion describe el proceso desde la descarga de la carpeta `MUJOCO R1` hasta la ejecucion de una rutina.

7.1 Paso 1: ubicar la carpeta descargada

Ejemplo recomendado:

```
mkdir -p "$HOME/Projects/unitree_lab"
mv "MUJOCO R1" "$HOME/Projects/unitree_lab/"
cd "$HOME/Projects/unitree_lab/MUJOCO R1"
```

Verificar contenido:

```
ls
ls 03_scripts_movimiento
ls 04_rutinas_txt
```

7.2 Paso 2: verificar Unitree MuJoCo

```
ls "$HOME/unitree_mujoco"
ls "$HOME/unitree_mujoco/simulate/build"
```

Si `$HOME/unitree_mujoco` no existe, se debe instalar primero Unitree MuJoCo siguiendo el procedimiento oficial del repositorio de Unitree. Este paquete no reemplaza la instalacion completa de MuJoCo.

7.3 Paso 3: instalar archivos R1 finales dentro de Unitree MuJoCo

Desde la carpeta MUJOCO R1:

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./07_instalacion/instalar_archivos_r1_en_unitree_mujoco.sh
```

Si el repositorio unitree_mujoco esta en otra ruta:

```
./07_instalacion/instalar_archivos_r1_en_unitree_mujoco.sh /ruta/a/unitree_mujoco
```

El script realiza backups automaticos antes de sobrescribir unitree_robots/r1 y simulate/config.yaml.

7.4 Paso 4: verificar unitree_sdk2py

```
python3 -c "import unitree_sdk2py; print('unitree_sdk2py OK')"
```

Si falla, instalar desde el repositorio local del SDK:

```
cd ~/unitree_sdk2_python  
pip3 install -e .
```

Despues verificar otra vez:

```
python3 -c "import unitree_sdk2py; print('unitree_sdk2py OK')"
```

7.5 Paso 5: abrir el simulador R1

Terminal 1:

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./run_simulador_r1.sh
```

Alternativamente, de forma manual:

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HOME/unitree_mujoco/simulate/mujoco/build/lib:/opt/  
unitree_robotics/lib  
cd "$HOME/unitree_mujoco/simulate/build"  
./unitree_mujoco
```

El archivo config.yaml debe contener:

```
robot: "r1"  
robot_scene: "scene.xml"  
domain_id: 0  
interface: "lo"  
print_scene_information: 1  
enable_elastic_band: 1
```

7.6 Paso 6: ejecutar una rutina

Con MuJoCo abierto y el R1 estable, abrir Terminal 2.

Para ejecutar siuu.txt:

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./run_siuu.sh
```

Para ejecutar dab.txt:

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./run_dab.sh
```

El script mostrara:

```
[ACCION] Verifica que el R1 este estable en MuJoCo. Presiona Enter para iniciar...
```

Verificar visualmente que el robot este estable y presionar Enter. El script ejecutara la secuencia y terminara.

7.7 Paso 7: validar sintaxis de una rutina

Antes de ejecutar una rutina nueva:

```
python3 -m json.tool 04_rutinas_txt/siuu.txt > /tmp/siuu_check.json  
python3 -m json.tool 04_rutinas_txt/dab.txt > /tmp/dab_check.json
```

7.8 Paso 8: capturar una nueva pose

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./capturar_pose_r1.sh
```

El script imprimira una tabla de articulaciones y un bloque JSON. Copiar el bloque dentro de una nueva rutina.

8. Seguridad Operacional

Advertencia critica. Este paquete controla articulaciones del robot R1 en simulacion mediante comandos de bajo nivel. Aunque el entorno descrito es MuJoCo, las rutinas y criterios de control no deben trasladarse directamente a robot fisico sin revision especifica, limites de torque, validacion de SDK fisico y protocolos de seguridad.

8.1 Condiciones antes de ejecutar

Antes de ejecutar cualquier rutina:

- Confirmar que MuJoCo esta abierto con el robot R1.
- Verificar que el robot esta estable antes de presionar Enter.
- Confirmar que se esta usando interface: "lo" en simulacion local.
- Confirmar que la rutina inicia y termina en pose segura.
- Validar el archivo con `python3 -m json.tool`.

8.2 Que no debe modificarse sin precaucion

- No cambiar el orden de sensores en `r1.xml`.
- No usar `Joint_index` como si fuera `Actuator_index`.
- No subir `Kp` o `ctrlrange` de forma global.
- No usar indices 26 o superiores.
- No eliminar la rampa de ganancias sin reemplazarla por otro mecanismo de suavizacion.

8.3 Detencion en simulacion

Si el robot empieza a oscilar o moverse de forma inesperada:

1. Detener el script Python con `Ctrl+C`.
2. Detener MuJoCo con `Ctrl+C`.
3. Revisar la rutina ejecutada.
4. Validar que no haya posiciones extremas o duraciones demasiado cortas.
5. Reiniciar MuJoCo antes de reintentar.

8.4 Consideraciones para robot fisico

Este documento no valida ejecucion fisica sobre R1 real. Para robot fisico se debe preparar una guia independiente que incluya interfaz de red real, topicos fisicos correctos, limites de torque reales, parada de emergencia fisica y validacion de que el robot no este caminando ni ejecutando otro modo.

9. Guia de Uso Rapido

9.1 Ejecucion rapida de siuu

1. Abrir la carpeta.

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"
```

2. Instalar archivos finales en `unitree_mujoco`.

```
./07_instalacion/instalar_archivos_r1_en_unitree_mujoco.sh
```

3. Abrir MuJoCo.

```
./run_simulador_r1.sh
```

4. En otra terminal, ejecutar la rutina.

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./run_siuu.sh
```

5. Esperar el mensaje de confirmacion y presionar `Enter`.

9.2 Ejecucion rapida de dab

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"
./run_dab.sh
```

10. Problemas Conocidos y Soluciones

Problema	Causa probable	Solucion
ModuleNotFoundError: unitree_sdk2py	SDK2 Python no instalado o no visible.	Ejecutar <code>pip3 install -e .</code> dentro del repositorio <code>unitree_sdk2_python</code> .
Error de libreria libmujoco.so	Falta LD_LIBRARY_PATH.	Exportar ruta a <code>simulate/mujoco/build/lib</code> .
MuJoCo abre otro robot	config.yaml no apunta a R1.	Verificar robot: "r1" y copiar configuracion final.
Robot cae al abrir simulador	No se esta usando unitree_mujoco correcto o falta elastic_band.	Verificar <code>enable_elastic_band: 1</code> .
No se recibe LowState	Dominio o interfaz incorrectos.	Usar <code>domain_id = 0</code> e <code>interface = 10</code> .
Advertencia <code>lo is not multicast-capable</code>	DDS sobre loopback.	En este flujo local puede aparecer y no necesariamente bloquea la ejecucion.
Robot se contorsiona al enviar comando	XML sin sensores articulares o sensores en orden incorrecto.	Usar el <code>r1.xml</code> final del paquete.
Brazos no alcanzan la pose	<code>ctrlrange</code> demasiado bajo.	Usar el <code>r1.xml</code> final con rangos corregidos.
Robot convulsiona o hace twitching	Ganancias o <code>ctrlrange</code> demasiado altos.	Usar configuracion final con rangos por grupo y rampa de ganancias.
Error de JSON	Coma faltante, llave mal cerrada o numero no valido.	Ejecutar <code>python3 -m json.tool archivo.txt</code> .
La rutina no se encuentra	Ruta incorrecta o carpeta distinta.	Ejecutar desde MUJOCO R1 o pasar ruta completa.
Movimiento muy brusco	Duraciones muy cortas o salto grande entre poses.	Agregar poses intermedias y aumentar duracion.
El robot no vuelve bien a pose segura	La rutina no termina con la pose segura estandar.	Copiar la pose segura final desde <code>r1_safe_pose_final.json</code> .

11. Extension y Mantenimiento

11.1 Agregar una nueva rutina

Para crear una nueva rutina:

1. Abrir MuJoCo.
2. Posicionar el R1 con sliders.
3. Capturar una pose.

```
cd "$HOME/Projects/unitree_lab/MUJOCO R1"  
./capturar_pose_r1.sh
```

Crear un archivo nuevo en 04_rutinas_txt/:

```
nano 04_rutinas_txt/nueva_pose.txt
```

Validar JSON:

```
python3 -m json.tool 04_rutinas_txt/nueva_pose.txt > /tmp/nueva_pose_check.json
```

Ejecutar:

```
python3 03_scripts_movimiento/r1_pose_player_mujoco.py 04_rutinas_txt/nueva_pose.txt
```

11.2 Recomendaciones para rutinas nuevas

- Usar duraciones de 1.5 a 3 segundos para transiciones amplias.
- No saltar directamente de pose segura a una pose extrema.
- Agregar poses intermedias para movimientos expresivos.
- Evitar cambios rapidos de una sola articulacion con amplitud grande.
- Mantener cabeza y muñecas con movimientos suaves.
- Probar primero con test_hold_actual.txt.

11.3 Validaciones despues de modificar

```
python3 -m py_compile 03_scripts_movimiento/r1_pose_player_mujoco.py  
python3 -m py_compile 03_scripts_movimiento/r1_capture_pose.py  
python3 -m json.tool 04_rutinas_txt/dab.txt > /tmp/dab_check.json  
python3 -m json.tool 04_rutinas_txt/siuu.txt > /tmp/siuu_check.json
```

12. Correcciones Tecnicas Realizadas

Problema encontrado	Solucion aplicada	Archivo final
R1 no se sostenia correctamente en escena normal.	Se uso Unitree MuJoCo con enable_elastic_band: 1.	02_config_simulador/config.yaml
Error de libreria MuJoCo al ejecutar.	Se agrego ruta de MuJoCo a LD_LIBRARY_PATH.	run_simulador_r1.sh
Confusion entre Joint_index y Actuator_index.	Se documento y uso mapeo por actuador.	r1_pose_player_mujoco.py, r1_capture_pose.py
LowState devolvia datos no articulares.	Se agregaron sensores jointpos, jointvel y actuatorfrc.	r1.xml
Brazos no subian ni alcanzaban poses amplias.	Se corrigio ctrlrange de actuadores.	r1.xml
Convulsion global al subir ctrlrange.	Se redujeron limites por grupo articular.	r1.xml
Twitching rapido en brazos.	Se redujeron ganancias y se agrego rampa de activacion.	r1_pose_player_mujoco.py
Rutinas sin estandar de seguridad.	Se definio pose segura obligatoria inicial y final.	r1_safe_pose_final.json, dab.txt, siuu.txt



13. Resumen de Actividades

Actividad	Estado	Observacion
Inspeccion del modelo R1	Completado	Se identifico mapeo real por actuadores y sensores faltantes.
Correccion de sensores en XML	Completado	Se agregaron sensores necesarios para LowState.
Correccion de limites de actuador	Completado	Se ajusto ctrlrange por grupo articular.
Ajuste de ganancias	Completado	Se logro estabilidad mediante ganancias suaves y rampa.
Validacion de test_hold_actual.txt	Completado	Robot estable.
Validacion de dab.txt	Completado	Rutina ejecutable desde el paquete.
Validacion de siuu.txt	Completado	Rutina validada como funcional y estable.
Organizacion de carpeta MUJOCO R1	Completado	Incluye modelo, scripts, rutinas, config y README.
Documentacion tecnica	Completado	Este documento funciona como guia reproducible.
Validacion en robot fisico	Pendiente	No incluida en el alcance de esta version. Requiere protocolo separado.

Cierre tecnico

El paquete MUJOCO R1 queda como una entrega reproducible para simulacion de poses del Unitree R1. La version final permite instalar los archivos corregidos sobre una instalacion existente de unitree_mujoco, ejecutar rutinas incluidas, capturar nuevas poses y extender el catalogo de movimientos manteniendo criterios minimos de seguridad, estabilidad y trazabilidad tecnica.